

我理解的BFF

我理解的BFF

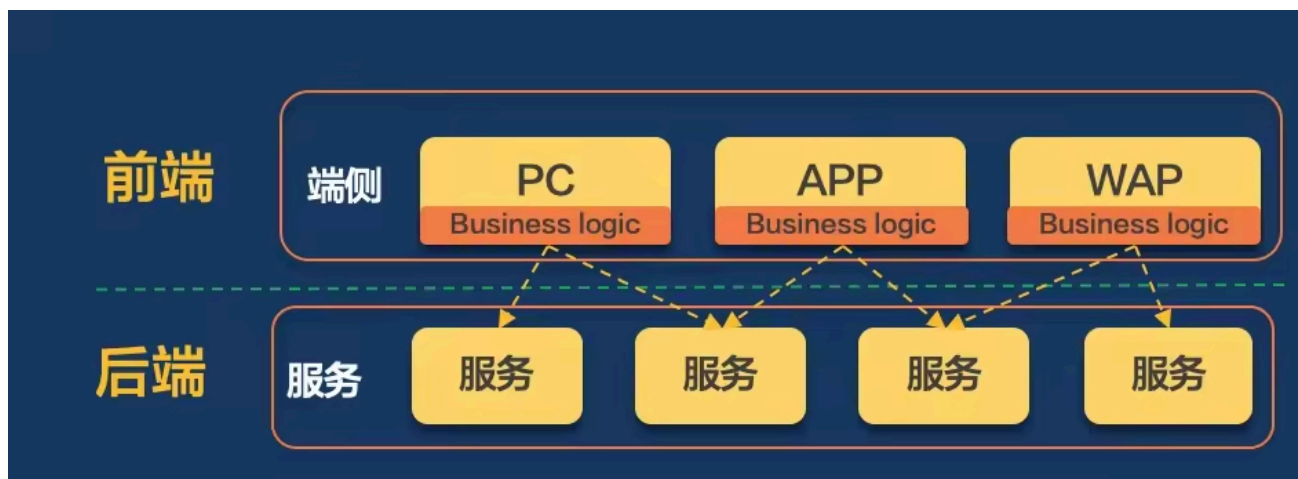
什么是BFF

BFF，即 Backend For Frontend（服务于前端的后端），起源于15年Sam Newman在他的博客中写的一篇文章——[Pattern: Backends For Frontends](#)。

BFF，其实简单来说，就是服务端设计 API 时考虑前端的使用，在服务端完成全部业务逻辑的处理，所以也称为用户体验适配器。

为什么需要BFF

一直以来，一直有一个问题萦绕在后端同学心中：服务端设计的API接口，到底是应该面向通用服务，还是要面向UI？



- 如果面向通用服务的话：那前端需要对接口进行二次加工，适配到UI层。这样，前端的展示组件中会耦合太多的业务逻辑，组件的通用性也会大大降低。
- 如果是面向UI的话：UI层是频繁变化的，后端需要为这些变化一一开发相应的服务，通常情况下，这些变化的服务复用性基本为零。后端的开发成本会大大增加。

这一时期内，后端和前端达成了某种意义上的一致：由于端侧请求的昂贵，后端负责了一部分的聚合逻辑，前端也维护了一部分的UI逻辑。这时的架构基本如下：



随着时代的发展，用户的接入形式也越来越多样。即使不同设备可能是实现相同的功能，但因为不同设备的特殊性，它们对服务端的API访问也各有其特点，需要区别处理。在这种情况下，后端不得不为每个端开发了对应的API，架构大致如下：



这种架构下，后端开发压力与日俱增，同时，这种情况下前后端各自负责了一部分UI逻辑，架构上也很不合理。

BFF理念：将内部复杂的微服务，适配成对各种不同用户体验（无线/Web/H5/第三方等）友好和统一的API。BFF相当于是前端和后端之间的一个中间层（适配层），作为前后端之间的桥梁，增加灵活性。BFF的提出，其实是开发人员在重新探索更合理的分层协作模式**。

BFF理念提出后，**端用户体验层->网关层->BFF层->微服务层**，是现代微服务架构的典型分层方式，这个架构能够灵活应对业务需求的变化，是一种支持创新的演化式架构。

BFF适配层应该具备哪些功能

总结下来，BFF层的主要是适配功能主要有：

- * 聚合
- * 多维度服务聚合：如APP端常见的个人页面里，常常需要将用户的个人信息接口 和 用户的偏好商品接口合并。
- * 单维度服务聚合：如商品的信息来自多个团队，基本信息、营销信息、DA离线统计信息、供应链信息等等。
- * 裁剪：
- * 字段精简：在APP端由于空间位置有限，只展示有限的字段
- * 隐私字段：后端微服务里返回的隐私字段不能在前端接口直接输出
- * 结构调整
- * 字段别名：如在前端组件里，商品标题的字段名为title，后端返回的字段名为subject
- * 将接口返回层次化结构拍平：如后端返回的结构是



复制代码

```
product{
  "subject":"001",
  "company":{
    "name":"公司名称",
    "url":"公司链接"
  }
}
```

而组件侧需要的结构是



复制代码

```
product{
  "subject":"001",
  "companyName":"公司名称",
  "companyUrl":"公司链接"
}
```

- MOCK
 - MOCK功能是解决前端、后端、算法间的互相依赖、互相等待问题的重要利器。提供与访问真实线上环境一致的MOCK服务API是前后端同学一直追求的梦想。

BFF的两种实现

虽然BFF是说“服务于前端的后端”，但这里的后端并不仅仅是java后端，也可以是Node层。

BFF的Nodejs实现

在大前端、全栈的思潮下，基于Nodejs的BFF层实现是最常见的。在这种架构下，前端通过Node直接调用服务端 RPC 接口拿到数据，按需加工消费数据，并实现UI层逻辑。



为了实现和java一样体验的RPC体验，前端同学开发了jar 的js proxy。前端同学只需在js里配置依赖的后端jar包，编译后，会自动生成js版本的rpc服务文档。

JAVA: 配置依赖jar包--》编译--》配置consumer--> 调用HSF服务

Node: 配置依赖jar包 --》编译 --》自动生成js文件（文档注释保留） --》调用HSF服务

在这种模式下，整体分工还比较清晰：后端通过 Java 等语言负责服务实现，理想情况下给前端提供的是基于领域模型的 RPC 接口，前端则在 BFF 层直接调用服务端 RPC 接口拿到数据，按需加工消费数据。

基于 Nodejs-BFF 模式的研发，很适合拥有前端技术背景的全栈型工程师。这种模式的好处很明显：

- * 沟通成本降低：

- * 展示层 和 BFF层 都是前端负责，属于团队内部。这样的话，可以让后端更加专注于业务逻辑和数据服务，不需要操心各种端的差异。我一直用一句话形容这种情况：前端自己开发业务API服务，然后自己测试，最后自己跟自己联调，调用该API服务，完成业务开发。

另外，也有人认为，这种情况下，后端可以专注于业务领域，更多从领域模型的视角去思考问题，页面视角的数据则交给前端型全栈工程师去搞定。领域模型与页面数据是两种思维模式，通过 BFF 可以很好地解耦开，让彼此更专业高效。

当然，这种模式下也有一定的问题：

- * 效能提升有限：还是基于代码开发的实现，这种实现方式跟B2B的Mbox基本类似，只是代码从java变成了js。
- * 学习成本增加：前端需要学习服务器的各种运维知识
- * 前端业务压力：前端负责的东西越来越多，带来的开发压力也成倍增加。

BFF的java端实现——基于GraphQL

BFF的java端实现与Nodejs的实现基本一致，这种架构下，前端定义数据Schema，对展示负责，后端服务于前端，提供完全适配数据Schema的接口，前后端通过数据模型交流：



GraphQL 作为一种 API 查询语句，于2015年被 Facebook 推出，主要是为了替代传统的 REST 模式。近两年越来越火。

我在前一篇文章“[三月大促国际站数据中心升级—全面拥抱GraphQL](#)”中也介绍了GraphQL的一些优势：按需返回、字段Mapping、服务聚合、通用脚本、配置MOCK 等，而这些功能都是BFF层最需要的。

因此后端通过引入GraphQL来完善BFF层也是近年来不错的选择，集团内也有很多团队在生产环境实践。

这种模式的好处也很明显：

* **开发效能提升**：BFF层是通过GraphQL的DSL描述的，而不是代码，前后端都可以编写GraphQL语句来完成适配逻辑。

* **服务聚合更专业**：后端同学对底层微服务更了解。前端同学由于缺乏对服务的了解，聚合适配出来的服务可能存在各种各样的问题，如性能较差。

当然这种架构也存在一定的问题：

* **DSL语言具有局限性**：很多复杂的业务逻辑无法在DSL层完成，还是需要后端服务代码层解决

我们的选择

基于后端服务的复杂性，我们团队目前选择的基于GraphQL的BFF架构。我们希望前端同学可以更专注于展示层，业务逻辑完全下沉到服务端。



另一方面，在未来，我们的目标是 **数据驱动组件**：前端同学开发纯UI层的组件，后端同学可以再保证数据Schema一致的情况下替换底层数据源，如果将页面上“历史浏览记录”切换为“猜你喜欢数据源”。这种情况下，前端同学根本无需感知到变化。

当然，上面任何一种方案都不是银弹，都需要根据自己团队、合作团队的情况合理选择。

最后的最后

小小思考，总结成文，希望可以帮助自己，也可以帮助他人理解BFF。当然，搜集资料仓促，自己理解也不够深刻，难免可能存在一些问题，万望告知~~~